WPF-Skelett — 4sight Hardening Generator

Lokale Windows-Desktop-App (.NET 8 / WPF). Sie lädt den **Katalog (JSON)**, zeigt die Einstellungen **nach Workload gruppiert** mit den Risiko-Zuständen **A/B/C** und erzeugt auf Knopfdruck das **PowerShell-Skript** (dieselbe geprüfte Preview/Apply/Revert-Logik wie das B-08-Beispiel, generalisiert über mehrere Einstellungen). Die App **wendet nichts an**.

Voraussetzungen

- Windows 10/11
- **.NET 8 SDK** (`dotnet --version` ≥ 8.0) — Download: https://dotnet.microsoft.com/download

Build & Start

```
cd HardeningGenerator
dotnet build            # kompilieren
dotnet run              # starten
```

Für eine verteilbare EXE:

```
dotnet publish -c Release -r win-x64 --self-contained false
# Ergebnis unter bin\Release\net8.0-windows\win-x64\publish\
```

Alternativ die `HardeningGenerator.csproj` einfach in Visual Studio 2022 öffnen und starten (F5).

Build-Fehler NU1100 (NuGet / .NET-Packs)

Meldung wie `error NU1100: "Microsoft.WindowsDesktop.App.Ref (= 8.0.x)" kann für "net8.0-windows7.0" nicht aufgelöst werden` . Das ist **kein Code-Fehler**, sondern **NuGet findet die .NET-8-Reference-Packs nicht**. Abhilfe in dieser Reihenfolge:

1. **`nuget.config` liegt bei** (im Projektordner, neben der `.csproj`) — sie setzt `nuget.org` als Quelle. Danach:

   ```
   dotnet restore
   dotnet build
   ```

   Der Build muss dafür **Internetzugang zu `api.nuget.org`** haben.

2. **SDK prüfen:** `dotnet --info` ausführen. Es muss ein **.NET 8 SDK** (nicht nur Runtime) gelistet sein. Achte auf die **Architektur / RID** (`win-x64` oder `win-arm64`): Auf einem **ARM64**-Windows muss das **ARM64-SDK** installiert sein — sonst scheitern genau die `win-arm64` -Host-Packs wie in der Fehlermeldung. Passendes SDK: https://dotnet.microsoft.com/download/dotnet/8.0

3. **Firewall/Proxy:** Ist `nuget.org` bei euch geblockt, in der `nuget.config` statt `nuget.org` **euren internen NuGet-Feed** eintragen (der muss die Microsoft-Packs spiegeln).

4. **Rein offline** ohne jeden Feed: dann müssen die **Targeting-Packs** lokal installiert sein (z. B. über das Visual-Studio-Installer-Workload „.NET-Desktopentwicklung”), damit kein Download nötig ist.

Wenn es danach noch hakt: schick mir die Ausgabe von `dotnet --info` — daran sieht man SDK, installierte Desktop-Runtime und Architektur, und ich sage dir genau, was fehlt.

Projektstruktur

```
HardeningGenerator/
  HardeningGenerator.csproj
  App.xaml / App.xaml.cs
  MainWindow.xaml / .cs         # UI + "Skript erzeugen"-Knopf
  Models/CatalogEntry.cs        # Abbild von catalog.schema.json
  Services/CatalogLoader.cs     # laedt alle catalog/*.json
  Services/ScriptGenerator.cs   # erzeugt das PowerShell-Skript
  ViewModels/MainViewModel.cs
```

```
ViewModels/SettingItemViewModel.cs
catalog/                      # der Katalog (wird neben die EXE kopiert)
  B-08_defender-av-realtime.json         (Klasse A, Gruppe defender-av)
  B-09_defender-av-cloudprotection.json  (Klasse A, Gruppe defender-av)
  B-10_defender-av-puaprotection.json    (Klasse A, Gruppe defender-av)
  B-11_defender-av-behaviormonitoring.json (Klasse A, Gruppe defender-av)
  B-13a_defender-av-scriptscan.json      (Klasse A, Gruppe defender-av)
  B-13b_defender-av-archivescan.json     (Klasse A, Gruppe defender-av)
  B-13c_defender-av-ioavprotection.json  (Klasse A, Gruppe defender-av)
  B-12_defender-networkprotection.json   (Klasse B, Audit, eigenstaendig)
  B-17_defender-controlledfolderaccess.json (Klasse B, Audit, eigenstaendig)
  A-04_ca-require-mfa_LOCKED.json         (Klasse C, gesperrt)
generated-example/
  Set-M365Hardening_EXAMPLE.ps1  # Beispiel-Ausgabe (B-08 + B-12)
```

Wie die Oberfläche die Leitplanken umsetzt

Risikoklasse	Darstellung	Wählbar?	Im Skript?
A – sicher	weiße Zeile	ja	ja
B – betriebs-/sicherheitskritisch	**gelbe** Zeile + Hinweis	ja, aber **nur die sichere Form** (z. B. Audit, unassigned)	ja (sichere Form)
C – aussperr-fähig	**graue** Zeile, Kontrollkästchen deaktiviert	**nein**	**nie** (doppelte Sperre: UI *und* Generator)

Das **Break-Glass-Feld** oben gibt die Konten vor; sie landen als `-BreakGlassUpn` im erzeugten Skript und werden dort bei der Ausführung blockiert.

Katalog erweitern — ohne Code-Änderung

Eine neue Einstellung = **eine neue JSON-Datei** im Ordner `catalog/` nach `catalog.schema.json`. Die App liest sie beim Start automatisch ein. Genau das ist der Sinn von „Katalog als Daten": Inhalt (und damit die Sicherheit) ist von der App-Logik getrennt.

Komplexe Einstellungen autoren (ASR, Firewall, BitLocker)

Bei Einstellungen mit regelspezifischen oder verschachtelten IDs nicht raten, sondern die **echte Struktur aus einem Test-Tenant exportieren** (read-only Helfer unter `tools/`):

```
# 1) In Intune ein Settings-Catalog-Profil mit den gewünschten Einstellungen anlegen
#    (z. B. ASR-Regeln auf 'Überwachen'/Audit), Name z. B. TEMP-ASR-Export, OHNE Zuweisung.
# 2) Exportieren:
.\tools\Export-IntuneConfigPolicy.ps1 -Name "TEMP-ASR-Export" -OutFile asr.json
```

Die `asr.json` enthält die exakten `settingDefinitionId`s und Choice-Values. Aus deren `settings[]`-Array wird 1:1 der `graph.createBody.settings` eines neuen Katalogeintrags.

Das erzeugte Skript

- Modi `-Mode Preview` (Standard, read-only) / `Apply` / `Revert`.
- Mehrere gewählte Einstellungen landen in **einem** Skript (Array `$Settings`), mit **vereinten Scopes** (Least Privilege), **Break-Glass-Sperre**, **Backup** je Einstellung und **unassigned**-Erzeugung.
- **„Bereits erfüllt"-Erkennung:** Die Preview scannt **alle** Intune-Profile und meldet, wenn der Soll-Wert schon durch ein anderes Profil gesetzt ist („BEREITS ERFÜLLT durch …") — dann ist ein eigenes Profil nicht nötig. Mit `-SkipSatisfied` überspringt auch `Apply` ein Profil, wenn **alle** seine Einstellungen bereits erfüllt sind (Standard: wie bisher anlegen).
- **Profil-Zusammenfassung (profileGroup):** Einträge mit gleicher `profileGroup.id` landen in **einem** tool-eigenen Profil (z. B. `4S-Defender-AV-Baseline` mit allen AV-Settings). Fremde Profile werden nie verändert. Re-Run = Profil ersetzen (löschen+neu); In-Place-Update folgt.
- Logik identisch zum geprüften B-08-Beispiel.

Status & Grenzen

- **Hier geprüft:** das **erzeugte PowerShell-Skript** ist gegen den PS-Parser validiert; die Klasse-C-Sperre im Generator ist verifiziert (A-04 erscheint nicht im Output). JSON + Schema sind valide.
- **Nicht hier prüfbar:** die WPF-App selbst kompiliert nur unter **Windows** (WPF ist Windows-only) — der C#-Code ist sorgfältig geschrieben, der erste `dotnet build` gehört auf eine Windows-Maschine.
- **Weiterhin offen (bewusst):** Graph-Setting-IDs im Test-Tenant verifizieren; echtes In-Place-Update statt Ersetzen; verwandte Einstellungen in ein Profil zusammenfassen; Pakete.

Nächste sinnvolle Schritte

1. `dotnet run` unter Windows, die drei Demo-Einträge durchklicken, ein Skript erzeugen und gegen einen **Test-Tenant** fahren (Preview → Apply → Assessment).
2. Restliche **A-Einträge der B-Sektion** als JSON in `catalog/` ergänzen.
3. **„Bereits erfüllt"-Erkennung** in die Preview einbauen.